Universidad de San Carlos de Guatemala

Ingeniería en Sistemas

Lenguajes Formales de Programación

Ing. Daniel Gonzales

Sección “A”



“PROYECTO 2 – MANUAL TÉCNICO, ANALIZADOR SINTÁCTICO PROMPTZAL”

202430221, Josué Adiel Oxlaj Paxtor

Quetzaltenango, 28 de octubre de 2026

Sumario

INTRODUCCIÓN.... 4

CONCEPTOS Y HERRAMIENTAS USADAS.... 5

REQUISITOS MÍNIMOS Y ENTORNO DE DESARROLLO.... 7

Requisitos de Hardware (Mínimos).... 7

Requisitos de Software.... 7

FUENTES DE DESCARGA.... 8

Descarga de Java JDK y Netbeans.... 8

Descarga de Graphviz y Dot.... 9

1. Acceder al portal oficial de Graphviz El usuario debe ingresar a la página web oficial (graphviz.org) y dirigirse a la pestaña de Download en el menú superior.... 9

Descargar jflex:.... 10

GUIÁ DE EJECUCIÓN.... 11

ESTRUCTURA DEL CÓDIGO.... 13

FUNCIONAMIENTO DEL CÓDIGO.... 15

ANALIZADOR LÉXICO.... 15

Tabla de Terminales.... 19

ANALIZADOR SINTÁCTICO.... 21

MODO PÁNICO (RECUPERACIÓN DE ERRORES SINTÁCTICOS).... 24

GRAMÁTICA BNF.... 25

DISEÑO DEL AST (ÁRBOL DE SINTAXIS ABSTRACTA).... 27

GENERACIÓN Y VISUALIZACIÓN GRÁFICA DEL AST.... 30

ANALIZADOR SEMÁNTICO..... 31

TablaSimbolos..... 32

AnalizadorSemantico..... 34

GENERACIÓN DEL PROMPTZAL..... 36

Ejemplo..... 37

COMPILADOR..... 39

REPORTES HTML..... 40

CONCLUSIÓN..... 41

INTRODUCCIÓN

En este documento se describe el funcionamiento del analizador sintáctico de promptzal correspondiente al proyecto 2 del curso de Lenguajes Formales de Programación que es la continuación del proyecto 1, donde se creo a mano un analizador léxico con su gramática regular y su modelo matemático AFD, ahora en este proyecto cumple el propósito del curso construir un compilador funcional que pueda realizar análisis léxico, sintáctico y semántico, analizando un archivo .pz y generar un prompt listo para utilizar en cualquier IA.

Para eso, el programa pasa el archivo por varias fases, primero el analizado léxico lo genera Jflex a partir de las expresiones regulares del lenguaje de promptzal, luego el analizado sintáctico el cual esta hecho con código puramente propio sin utilizar ninguna herramienta o librería externa, esta escrito a mano como un parser descendente recursivo, con un método por cada no terminal de la gramática, ademas que con el método modo pánico es capaz de leer y reportar varios errores en una sola fila, mientras el parser avanza y reconoce el lenguaje, se va armando el árbol de sintaxis abstracta (AST), que después se recorre para generar la tabla de símbolos , validar lo analizado y así generar el prompt final listo para copiar y pegar.

El proyecto esta hecho en Java con Netbeans como IDE y Maven, ademas para la generación del AST y el AFD se utilizo Graphviz con su respectivo código Dot. En las siguientes secciones se explica cómo está hecha cada parte: la tabla de terminales, la gramática, el archivo de JFlex, el diseño del AST y de la tabla de símbolos, el manejo de errores y la generación del promptzal. Tambien se resaltan las decisiones de diseño y algunas limitaciones que el programa tiene para tener todo bien descrito.

CONCEPTOS Y HERRAMIENTAS USADAS

Java: Java es un lenguaje de programación de alto nivel, orientado a objetos y diseñado para tener las menores dependencias de implementación posibles. Su filosofía *"Write Once, Run Anywhere"* (Escribe una vez, ejecútalo en cualquier lugar) permite que este proyecto sea portable.

Para este desarrollo se utilizó específicamente el JDK (Java Development Kit) versión 21, que es el conjunto de herramientas necesarias para compilar y convertir el código fuente (.java) en bytecode ejecutable por la JVM (Java Virtual Machine).

Java Swing: Java Swing es una biblioteca de Java para crear interfaces gráficas de usuario (GUI) en aplicaciones de escritorio.

Forma parte de Java Foundation Classes (JFC) y está incluida en el JDK desde Java 1.2, por lo que no requiere instalación adicional.

Apache NetBeans: Apache NetBeans es un Entorno de Desarrollo Integrado (IDE) de código abierto. Se eligió esta herramienta porque proporciona un buen soporte para la gestión de proyectos basados en Maven, facilitando la depuración, la organización de paquetes y la compilación automatizada del código. NetBeans actúa como el ecosistema donde se integran el editor de texto, el compilador y las herramientas de salida de consola.

Maven: Apache Maven es una potente herramienta de gestión y automatización de compilación, utilizada principalmente en proyectos Java. Facilita la estructura del proyecto, gestiona dependencias (bibliotecas externas) automáticamente y estandariza el ciclo de vida del desarrollo (compilación, pruebas, empaquetado) mediante un archivo de configuración central llamado pom.xml.

Pom.xml: POM significa Project Object Model (Modelo de Objetos de Proyecto) y es el archivo de configuración central (pom.xml) en proyectos Java gestionados con Apache Maven. Define los metadatos del proyecto, gestiona las dependencias (librerías externas), complementos (plugins) y la

configuración de construcción (build) para compilar, probar y empaquetar la aplicación de forma automática.

AFD: modelo matemático que procesa cadenas de símbolos paso a paso y decide si acepta o rechaza cada secuencia según las reglas de transición.

Gramática de un lenguaje regular: gramática regular es un tipo de regla matemática que sirve para generar un lenguaje regular, que es el grupo más sencillo de palabras dentro de la teoría de la computación.

Jerarquía de Chomsky: La jerarquía de Chomsky es una clasificación de cuatro niveles de las gramáticas formales que generan lenguajes formales, descrita por el lingüista Noam Chomsky en 1956.

Graphviz y Dot: Graphviz es un programa de código abierto que sirve para crear diagramas y redes a partir de instrucciones de texto, mientras que DOT es el lenguaje de programación en formato de texto plano que se utiliza para describir esos gráficos

Git: Git es un programa que se instala en la computadora para guardar el historial de los cambios dentro del código.

GitHub: GitHub es una página web en la nube que guarda copias de proyectos para compartirlos con otras personas.

Jflex: JFlex es un generador de analizadores léxicos (también llamados escáneres o lexers) para el lenguaje de programación Java. Escrito también en Java, su función principal es leer un flujo de texto de entrada, identificar patrones de caracteres específicos mediante expresiones regulares y agruparlos en unidades con significado conocidas como tokens (como palabras clave, operadores, números o identificadores).

REQUISITOS MÍNIMOS Y ENTORNO DE DESARROLLO

Para asegurar la correcta ejecución y posible modificación del sistema, el entorno debe cumplir con las siguientes especificaciones técnicas.

Requisitos de Hardware (Mínimos)

- **Procesador**: Arquitectura x64 de 2.0 GHz o superior.
- **Memoria RAM:** 8 GB (Recomendado para el manejo de IDEs pesados).
- **Almacenamiento:** 1 GB de espacio libre para el proyecto y dependencias de Maven.
- **Periféricos:** Teclado y monitor (Interfaz basada en consola).
- **Resolución de pantalla mínima**: resolución mínima de 1280x720, para no tener problemas con la interfaz gráfica.

Requisitos de Software

- **Sistema Operativo:** Windows 10/11, distribuciones de Linux (Ubuntu/Debian) o macOS.
- Java Development Kit (JDK): Versión 21 (LTS).
- **IDE:** Apache NetBeans 21 o superior.
- **Graphviz :** Versión recomendada (Graphviz 9.0 o superior).
- **Apache Maven:** al tener Apache NetBeans KIT versión 21, ya lo tendrá incorporado.
- **Git**: versión de control, necesario para el historial de commits.
- **Navegador web**: para visualizar los reportes HTML generados (tokens, errores, estadísticas), no importa cuál.

- **Sistema de visualización de imágenes:** Sistema que la computadora tiene por defecto para visualizar imágenes, no importa cual es, esto es importante para poder ver el AFD que estará en este proyecto.
- **IDE** : se requiere de un IDE para poder acceder al codigo fuente, se recomienda (Netbeans o Eclipse) , ya que estos estas desarrollados para un mejor trabajo con el lenguaje java, pero cualquier IDE es suficiente.

FUENTES DE DESCARGA

Descarga de Java JDK y Netbeans

1. **Verificar la instalación previa de Java (JDK):** NetBeans requiere un Kit de Desarrollo de Java (JDK) compatible para ejecutarse. El usuario debe comprobar si ya cuenta con una versión adecuada (por ejemplo, JDK 17 o superior) o descargarla previamente desde sitios como Oracle o Eclipse Adoptium (Temurin) **JDK 21:** https://www.oracle.com/java/technologies/downloads/.
2. **Acceder al sitio web oficial:** El usuario debe ingresar al portal oficial de Apache NetBeans Java **Apache NetBeans:** https://netbeans.apache.org/front/main/download/,desde cualquier navegador web.
3. **Ir a la sección de descargas:** En la página principal, se debe hacer clic en el botón o pestaña de Download para visualizar la versión estable más reciente disponible.
4. **Seleccionar el instalador para el sistema operativo:** Dentro de la sección de instaladores binarios, el usuario debe elegir el archivo correspondiente a su plataforma: Instalador .exe para Windows, Archivo .pkg o .dmg para macOS, Script .sh o paquete .deb / .rpm para distribuciones de Linux.

5. **Elegir un servidor espejo (Mirror):** El sitio web sugerirá un enlace de descarga sugerido por Apache; el usuario debe hacer clic en el enlace principal para iniciar la transferencia del archivo instalador.
6. **Ejecutar el instalador:** Una vez completada la descarga, el usuario debe abrir el archivo ejecutable con permisos de administrador y seguir las instrucciones del asistente en pantalla para configurar la ruta de instalación y vincular la ruta del JDK correspondiente.

Descarga de Graphviz y Dot

1. **Acceder al portal oficial de Graphviz** El usuario debe ingresar a la página web oficial (graphviz.org) y dirigirse a la pestaña de **Download** en el menú superior.
2. **Seleccionar el instalador según el sistema operativo** Dentro de la sección de descargas, el usuario debe ubicar su plataforma:

Windows: Descargar el paquete instalador ejecutable (generalmente graphviz-install-X.X.X-win64.exe).

Linux (Debian/Ubuntu): Se puede instalar directamente desde la terminal mediante el gestor de paquetes ejecutando sudo apt install graphviz.

macOS: Se puede instalar fácilmente utilizando Homebrew con el comando brew install graphviz.

3. **Ejecutar el asistente de instalación (en Windows)** El usuario debe iniciar el archivo .exe descargado y seguir las indicaciones en pantalla para aceptar la licencia y definir el directorio de instalación.

4. **Habilitar la adición al PATH durante la instalación** En el asistente de Windows, es crucial que el usuario marque la casilla que indica **"Add Graphviz to the system PATH for all users"** (o para el usuario actual). Esto permitirá ejecutar el comando dot desde cualquier terminal o programa externo.
5. **Configurar el PATH manualmente (si no se marcó en el paso anterior)** Si el comando no se reconoce de forma automática, el usuario debe:

Abrir las **Variables de entorno del sistema** en el panel de control.

Editar la variable Path en las variables de entorno.

Añadir la ruta completa a la carpeta bin de Graphviz (por ejemplo, C:\Program Files\Graphviz\bin).

6. **Comprobar la instalación de la herramienta dot** Para verificar que la herramienta esté lista para su uso, el usuario debe abrir una terminal (PowerShell, CMD o bash) y ejecutar: *dot -V* El comando debe imprimir en pantalla la versión instalada de Graphviz.

Descargar jflex:

El usuario puede descargar e instalar JFlex siguiendo este procedimiento
Visita el sitio web oficial de JFlex en jflex.de y descarga el archivo comprimido más reciente desde la sección de descargas.

Descomprimir el archivo en una carpeta del disco duro como por ejemplo en un directorio de herramientas del sistema operativo.

Agregar la ruta de la carpeta bin al PATH del sistema y crea la variable JFLEX_HOME apuntando a la carpeta principal para usar el comando desde la terminal.

Si se utiliza Maven añadir el jflex maven plugin al archivo pom xml para que el sistema realice la descarga de forma automática.

Abrir una terminal y ejecuta el comando jflex version para verificar que el sistema reconoce la instalación de manera correcta.

Como implementar Jflex en Netbeans o cualquier IDE:

La forma más eficiente y estándar consiste en delegar la tarea a Maven o Gradle ya que cualquier IDE moderno como IntelliJ IDEA NetBeans o Eclipse detecta estas configuraciones automáticamente

Si se utiliza Maven el desarrollador añade el jflex maven plugin en el archivo pom xml y el IDE compilará los archivos flex al compilar el proyecto

Si se utiliza Gradle se incluye el plugin de JFlex en el archivo build gradle y el IDE se encarga de la generación de código de forma transparente.

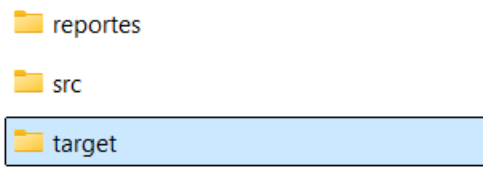
GUIÁ DE EJECUCIÓN

Con los requerimientos mínimos y las herramientas necesarias instaladas ya es posible ejecutar el programa en el equipo sin ningún tipo de problema a continuación se muestra el paso a paso de como poder ejecutar el proyecto correctamente:

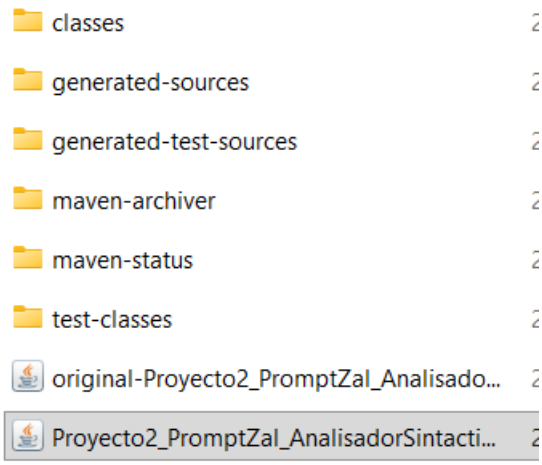
Con el archivo .zip se extrae en cualquier carpeta dentro del sistema preferiblemente en descargas, documentos o alguna carpeta especial.



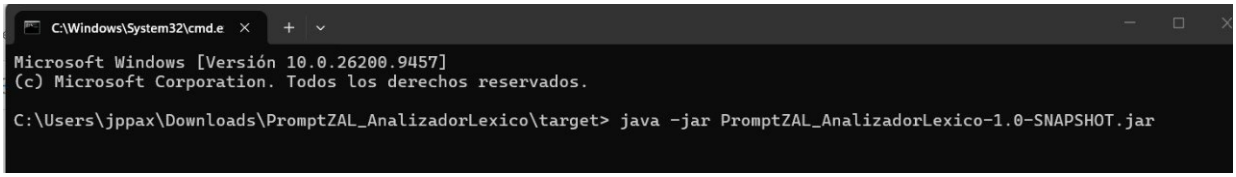
Es importante descomprimir la carpeta para poder acceder al contenido de la carpeta donde se encuentra el código fuente y el proyecto.



En la carpeta target se encuentra el proyecto, al acceder a la carpeta es posible visualizar el archivo .jar que contiene el programa, para acceder a el simplemente se puede hacer click dos veces sobre el archivo sin el sufijo original o también se puede acceder desde la terminal.



Para acceder con terminal se debe escribir CMD, en la ruta de la carpeta una vez escrito se presiona enter y se abrirá la terminal del equipo, escribiendo la siguiente ruta: **java -jar Proyecto2_PromptZal_AnalisadorSintactico-1.0-SNAPSHOT.jar** y presionando enter nuevamente se puede acceder al programa PROMPT_ZAL.



Al configurar el Maven Shade Plugin-en el pom.xml, Maven genera dos archivos .jar: uno con el sufijo original- (el binario limpio base) y otro sin sufijo (el paquete completo con todo integrado). Esta separación se realiza para preservar el flujo de compilación estándar de Maven, evitar conflictos y permitir una distribución autónoma.

ESTRUCTURA DEL CÓDIGO

El código esta dividido en paquetes para mantener un estilo limpio y aplicando los conceptos básicos de POO .

Paquete	Que hace	Clases principales
lexer	Convierte el texto del .pz en tokens. El lexer lo genera JFlex a partir de Lexer.jflex	Token, TipoToken, AnalizadorLexico, Lexer (generada)
Parser	Revisa que los tokens sigan la gramática, con un método por no terminal, recuperación en modo pánico y construyendo el AST	parser
ast	Guarda la estructura del programa en memoria. Son solo clases de datos, sin lógica	Programa, Agente, Variable, Comando, Conector, Directiva, Referencia, Expresion, Termino y sus cuatro subclases
semantico	Recorre el AST, llena la tabla de símbolos y valida el significado: variables sin declarar, agentes duplicados, EJECUTAR y EXPORTAR inválidos	AnalizadorSemantico, TablaSimbolos, Simbolo, TipoSimbolo

generador	Recorre el AST y escribe el promptzal en texto	GeneradorPromptzal
errores	Una sola clase compartida donde las tres fases registran sus errores	ErrorCompilacion
compilador	Une las fases en orden (léxico, parser, semántica y generador) y entrega el resultado a la interfaz	Compilador, ResultadoCompilacion
reportes	Genera los reportes HTML de errores, símbolos y AST, con un estilo compartido	ReporteErrores, ReporteSimbolos, ReporteAst, Html
graphviz	Convierte el AST a formato DOT y ejecuta Graphviz para dibujarlo como imagen	GeneradorDotAst, MostrarGraphviz
afd	Define las transiciones del AFD y genera su diagrama	TablaTransiciones, GeneradorDot
vista	La interfaz tipo IDE: editor con pestañas, números de línea, coloreado, paneles de resultados y reportes	VentanaPrincipal, PestanaEditor, EditorCodigo, ColoreadorSintaxis, Tema, entre otras

Todos los paquetes y clases cumplen con varias tareas cada uno pero trabajando en conjunto para poder realizar 4 funciones esenciales para el funcionamiento del promptzal el análisis léxico, sintáctico, semántico y el procesamiento de errores.

FUNCIONAMIENTO DEL CÓDIGO

En esta sección se describe como funciona el cogido, la manera en las que todas las partes funcionan y trabajan en conjunto para realizar los análisis, léxicos, sintácticos y semánticos de un archivo pz. Como se reconocen los tokens, la gramática y como el código es capaz de generar los reportes para que el usuario que haga uso del analizador sea capaz de entender y ver como funciona la gramática.

ANALIZADOR LÉXICO

El analizador léxico lee el archivo .pz carácter por carácter y lo convierte en una lista de tokens. Cada token guarda su tipo, el texto que lo formó (el lexema) y la fila y la columna donde empieza, contadas desde 1. Los espacios, fabulaciones, saltos de línea y comentarios no le sirven al parser, así que se descartan. Al final de la lista se agrega un token EOF. Si aparece algo que no se reconoce, se anota como error léxico y el análisis sigue.

El analizador lexico es el único de este compilador que esta hecho con la implementación de una herramienta externa pero que sigue siendo perteneciente al entorno de java, para la creación de este lexer se utilizo jflex.

Se creo un archivo especifico Lexer.jflex desde el IDE Netbeans utilizando la opcion de creación de clases con caracterizara other file (otro archivo) donde se declararon los tokens y Jflex lo convierte en clase .java con el comando java -jar jflex-full-1.9.1.jar Lexer.jflex que se ejecuta desde la carpeta del lexer.

Lexer.flex

En esta sección se configura la clase pública Lexer para leer texto en formato Unicode, llevar el conteo de filas y columnas, devolver objetos de tipo Token y manejar un estado exclusivo llamado COMENTARIO. Dentro del bloque %{ ... %} se declara la lista donde se guardan los errores léxicos encontrados, se crean los métodos token() y error() (que le suman 1 a yyline y yycolumn para que las posiciones empiecen en 1 y no en 0), y se definen las variables comFila y comColumna para registrar dónde inicia un comentario de bloque.

```
%public
%class Lexer
%unicode
%line
%column
%type Token
%xstate COMENTARIO

%{
    private List<ErrorCompilacion> errores;

    public Lexer(java.io.Reader in, List<ErrorCompilacion> errores) {
        this(in);
        this.errores = errores;
    }

    private Token token(TipoToken tipo) {
        return new Token(tipo, yytext(), yyline + 1, yycolumn + 1);
    }

    private void error(String descripcion) {
        errores.add(ErrorCompilacion.lexico(descripcion, yyline + 1, yycolumn + 1));
    }

    private int comFila;
    private int comColumna;
%}
```

```
47    "@modelo" | "@rol" | "@formato"   { return token(TipoToken.DIRECTIVA); }
48
49    "AGENTE"      { return token(TipoToken.AGENTE); }
50    "contexto"    { return token(TipoToken.CONTEXTO); }
51    "variable"    { return token(TipoToken.VARIABLE); }
52    "EJECUTAR"    { return token(TipoToken.EJECUTAR); }
53    "EXPORTAR"    { return token(TipoToken.EXPORTAR); }
54    "CARGAR"      { return token(TipoToken.CARGAR); }
55
56    "PREGUNTAR" | "GENERAR" | "RESUMIR" | "ANALIZAR" |
57    "TRADUCIR" | "CLASIFICAR" | "EXTRAER"   { return token(TipoToken.COMANDO_IA); }
58
59    "SOBRE" | "DESDE" | "EN" | "COMO"   { return token(TipoToken.CONECTOR); }
60
61    "->"   { return token(TipoToken.FLECHA); }
62    "="    { return token(TipoToken.IGUAL); }
63    "+"    { return token(TipoToken.MAS); }
64    "{"    { return token(TipoToken.LLAVE_A); }
65    "}"    { return token(TipoToken.LLAVE_C); }
66    "("    { return token(TipoToken.PAR_A); }
67    ")"    { return token(TipoToken.PAR_C); }
68    ","    { return token(TipoToken.COMA); }
69
70    {ID}        { return token(TipoToken.ID); }
71    {NUMERO}    { return token(TipoToken.NUMERO); }
72    {CADENA}    { return token(TipoToken.CADENA); }
73
74    {ESPACIO} | {COM_LINEA}   { }
75
76    "/*"   { comFila = yyline + 1; comColumna = yycolumn + 1; yybegin(COMENTARIO); }
77
78    <COMENTARIO> {
79        "*/"    { yybegin(YYINITIAL); }
80        [^]     { }
81        <<EOF>> {
82            errores.add(ErrorCompilacion.lexico("Comentario de bloque sin cerrar", comFila, comColumna));
83            yybegin(YYINITIAL);
84            return new Token(TipoToken.EOF, "", yyline + 1, yycolumn + 1);
85        }
86    }
87
```

En esta parte se definen las reglas que transforman el texto de entrada en tokens. , los conectores y los símbolos de puntuación y operadores (->, =, +, {, }, (,), ,). Estas reglas se colocan antes de {ID} para que JFlex reconozca primero las palabras del lenguaje y no las tome como identificadores comunes.

Después se evalúan las expresiones para identificadores ({ID}), números ({NUMERO}) y textos entre comillas ({CADENA}), mientras que los espacios en blanco ({ESPACIO}) y comentarios de una línea ({COM_LINEA}) se ignoran con una acción vacía { }. Por último, al encontrar /* se guarda la fila y columna de inicio y el analizador pasa al estado <COMENTARIO>, donde descarta cualquier carácter ([^]) hasta leer */ y volver al estado normal (YYINITIAL); si el archivo termina antes (<<EOF>>), registra el error "Comentario de bloque sin cerrar" con la posición guardada y devuelve el token EOF.

Lexer.java

Esta clase Lexer.java es la que se genera después de compilar el archivo Lexer.jflex desde la consola con JFlex 1.9.1, convirtiendo las reglas léxicas a código Java para poder usar el analizador dentro del proyecto.

```
// DO NOT EDIT
// Generated by JFlex 1.9.1 http://jflex.de/
// source: Lexer.jflex

package com.cunoc.proyecto2_promptzal_analisadorsintactico.lexer;

import java.util.List;
import com.cunoc.proyecto2_promptzal_analisadorsintactico.errores.ErrorCompilacion;

//aqui estan los %%

@SuppressWarnings("fallthrough")
public class Lexer {
```

Token

La clase Token sirve como plantilla para guardar la información de cada palabra o símbolo que el analizador léxico encuentra en el archivo de entrada. Guarda cuatro datos que no cambian (final): el tipo de token (TipoToken), el texto exacto leído (lexema), y la fila y columna donde apareció, con sus métodos get para que el parser pueda consultarlos durante el análisis.

```java
public class Token {
    private final TipoToken tipo;
    private final String lexema;
    private final int fila;
    private final int columna;

    public Token(TipoToken tipo, String lexema, int fila, int columna) {
        this.tipo = tipo;
        this.lexema = lexema;
        this.fila = fila;
        this.columna = columna;
    }

    public TipoToken getTipo() { return tipo; }
    public String getLexema() { return lexema; }
    public int getFila() { return fila; }
    public int getColumna() { return columna; }
}
```

TipoToken

El enum TipoToken lista las 21 categorías de símbolos terminales que reconoce el lenguaje. Están separados en tres grupos: las directivas , palabras reservadas , los signos, operadores , y los valores variables junto con EOF para indicar que terminó el archivo.

```java
public enum TipoToken {
    DIRECTIVA,
    AGENTE,
    CONTEXTO,
```

Tabla de Terminales

TipoToken	Expresión Regular
DIRECTIVA	"@modelo" | "@rol" | "@formato"
AGENTE	“AGENTE”

CONTEXTO	“contexto”
VARIABLE	“variable”
EJECUTAR	“EJECUTAR”
EXPORTAR	“EXPORTAR”
CARGAR	“CARGAR”
COMANDO_IA	"PREGUNTAR" | "GENERAR" | "RESUMIR" | "ANALIZAR" | "TRADUCIR" | "CLASIFICAR" | "EXTRAER"
CONECTOR	"SOBRE" | "DESDE" | "EN" | "COMO"
FLECHA	"->"
IGUAL	“=”
MAS	“+”
LLAVE_A	“{”
LLAVE_C	“}”
PAR_A	“(”
PAR_C	“)”
COMA	“,”
ID	[\p{L}_]([\p{L}_]|[0-9])*
NUMERO	[0-9]+(\.[0-9]
CADENA	\"[^\"\r\n]*\"

EOF	<<EOF>>

ANALIZADOR SINTÁCTICO

El analizador sintáctico se construyó de forma manual usando el método de descenso recursivo, ya que la gramática del lenguaje es LL(1) y no tiene recursividad por la izquierda. Funciona pidiendo tokens uno por uno al analizador léxico y comparando el token actual con el que se espera según las reglas de la gramática: cada símbolo no terminal tiene su propio método en la clase Parser, y las listas repetitivas (como varias directivas, agentes, variables, comandos o conectores) se recorren con ciclos while. Mientras valida el orden de los tokens, va creando y enlazando los nodos del Árbol de Sintaxis Abstracta (AST). Si encuentra un token que no corresponde, registra el error sintáctico y aplica recuperación en modo pánico, descartando tokens hasta hallar uno de sincronización (como el inicio de otro comando, otra variable, otro agente o una llave de cierre) para seguir revisando el resto del archivo sin detener el programa.

Parser

```
63      private Token match(TipoToken esperado) {
64          if (verifica(esperado)) return avanzar();
65          throw error(esperado.name());
66      }
67
68      private ErrorSintactico error(String esperado) {
69          Token t = actual();
70          if (pos != posUltimoError) {
71              errores.add(ErrorCompilacion.sintactico(describir(t), esperado, t.getFila(), t.getColumna()))
72              posUltimoError = pos;
73          }
74          return new ErrorSintactico();
75      }
76
77      private void sincronizar() {
78          while (!SINCRONIZACION.contains(actual().getTipo())) avanzar();
79          posUltimoError = pos;
80      }
```

Metodos match, error, sincronizar:

El método match comprueba si el token actual coincide con el tipo esperado; si es así, avanza al siguiente y lo devuelve, pero si no coincide, lanza la excepción interna ErrorSintactico. Antes de lanzarla, el método error guarda el fallo en la lista indicando qué token se encontró, cuál se esperaba y su fila y columna, revisando primero que pos != posUltimoError para no registrar dos veces el mismo error en la misma posición. Cuando ocurre ese fallo, los bloques del parser llaman a sincronizar(), que aplica la recuperación en modo pánico descartando tokens con avanzar() hasta encontrar un token seguro del conjunto SINCRONIZACION (AGENTE, LLAVE_C, COMANDO_IA, VARIABLE, EJECUTAR, EXPORTAR o EOF) y actualiza posUltimoError para seguir analizando el resto del archivo.

Metodo Parser

```
    public Parser(List<Token> tokens, List<ErrorCompilacion> errores) {
        this.tokens = tokens;
        this.errores = errores;
    }
```

El constructor de la clase Parser recibe la lista de tokens generada por el analizador léxico y la lista compartida de errores del compilador, guardándolas en sus atributos internos para recorrer los tokens durante el análisis y registrar cualquier fallo sintáctico en esa misma lista.

agente

```
272    private Agente agente() {
273        Token inicio = match(TipoToken.AGENTE);
274        String nombre = null;
275        String contexto = null;
276        try {
277            nombre = match(TipoToken.ID).getLexema();
278            match(TipoToken.LLAVE_A);
279            contexto = contexto();
280        } catch (ErrorSintactico e) {
281            sincronizar();
282        }
283        List<Variable> variables = variables();
284        List<Comando> comandos = comandos();
285        cerrarAgente(comandos);
286        return new Agente(nombre, contexto, variables, comandos, inicio.getFila(), inicio.getColumna());
287    }
```

El método agente() lee la estructura completa de un bloque AGENTE. Primero consume la palabra reservada AGENTE y luego intenta leer dentro de un try-catch el identificador (ID), la llave de apertura (LLAVE_A) y el contexto(). Si ocurre un error sintáctico en esa cabecera, las variables nombre o contexto quedan en null y se llama a sincronizar() para no perder el resto del bloque y poder seguir leyendo su lista de variables() y comandos(). Por último, invoca a cerrarAgente(comandos) para asegurar la llave de cierre y devuelve el nodo Agente con su posición de inicio.

Función de los métodos de las reglas gramaticales:

Cada símbolo no terminal de la gramática tiene su propio método en la clase Parser. Cuando una regla tiene un orden fijo de elementos (como comando, variable o contexto), el método llama a match() paso por paso para consumir cada token esperado y construir el nodo del AST. Cuando una regla permite repetir elementos cero o más veces (como las listas de directivas, variables, masComandos, conectores o masTerminos), se usa un ciclo while (verifica(...)) que sigue leyendo mientras aparezca el token inicial de esa estructura. Por último, cuando una regla tiene varias opciones distintas (como en termino()), se usa un switch con el tipo del token actual (actual().getTipo()) para decidir qué camino tomar: si lee CADENA, ID, NUMERO o CARGAR, crea el nodo correspondiente, y si llega cualquier otro token entra al default y lanza un error sintáctico.

```
200     // <comando> -> COMANDO_IA <expresion> <conectores> FLECHA ID
201     private Comando comando() {
202         Token inicio = match(TipoToken.COMANDO_IA);
203         Expresion argumento = expresion();
204         List<Conector> conectores = conectores();
205         match(TipoToken.FLECHA);
206         Token destino = match(TipoToken.ID);
207         return new Comando(inicio.getLexema(), argumento, conectores,
208                 destino.getLexema(), inicio.getFila(), destino.getFila());
209     }
210
```

MODO PÁNICO (RECUPERACIÓN DE ERRORES SINTÁCTICOS)

Para el manejo de errores durante el análisis sintáctico hay dos métodos importantes que se encargan de trabjar en ellos que son sincronizar() y sincronizarPrograma()

El método sincronizar() se encarga de descartar los tokens que causan problemas hasta encontrarse con un símbolo seguro dentro del bloque del agente, mientras que sincronizarPrograma()

funciona a nivel general del archivo para saltar hasta el inicio de un nuevo componente importante como un agente o bloque de ejecución.

```
    private void sincronizar() {
        while (!SINCRONIZACION.contains(actual().getTipo())) avanzar();
        posUltimoError = pos;
    }

    private void sincronizarPrograma() {
        while (!SINCRONIZACION_PROGRAMA.contains(actual().getTipo())) avanzar();
        posUltimoError = pos;
    }
```

En lugar de detener la compilación por completo al primer error, el parser lanza una excepción interna (ErrorSintactico), la atrapa en el bloque donde ocurrió y llama al método sincronizar().

ErrorCompilacion

La clase ErrorCompilacion es la encargada de estandarizar, almacenar y gestionar todos los errores que ocurren durante las distintas fases del proceso de compilación (léxico, sintáctico y semántico). Su diseño utiliza un patrón de fábrica mediante métodos estáticos, lo que obliga a declarar los errores de forma limpia y controlada, asegurando que cada incidente reporte su ubicación exacta (fila y columna) y la categoría a la que pertenece.

```
    public enum Tipo { LEXICO, SINTACTICO, SEMANTICO }

    private final Tipo tipo;
    private final String descripcion;
    private final int fila;
    private final int columna;
    private final String encontrado;
    private final String esperado;
```

```java
    // Constructor bloqueado para obligar al programa a usar los metodos estaticos de abajo
    private ErrorCompilacion(Tipo tipo, String descripcion, int fila, int columna,
                             String encontrado, String esperado) {
        this.tipo = tipo;
        this.descripcion = descripcion;
        this.fila = fila;
        this.columna = columna;
        this.encontrado = encontrado;
        this.esperado = esperado;
    }
```

GRAMÁTICA BNF

<programa> -> <directivas> <agentes> <ejecuciones> <exportar> EOF

<directivas> -> DIRECTIVA CADENA <directivas> | ε

<agentes> -> <agente> <mas_agentes>

<mas_agentes> -> <agente> <mas_agentes> | ε

<agente> -> AGENTE ID LLAVE_A <contexto> <variables> <comandos> LLAVE_C

<contexto> -> CONTEXTO IGUAL CADENA

<variables> -> VARIABLE ID IGUAL <expresion> <variables> | ε

<comandos> -> <comando> <mas_comandos>

<mas_comandos> -> <comando> <mas_comandos> | ε

<comando> -> COMANDO_IA <expresion> <conectores> FLECHA ID

<conectores> -> CONECTOR <expresion> <conectores> | ε

<expresion> -> <termino> <mas_terminos>

<mas_terminos> -> MAS <termino> <mas_terminos> | ε

<termino> -> CADENA | ID | NUMERO <unidad> | <cargar>

<unidad> -> ID | ε

<cargar> -> CARGAR PAR_A CADENA PAR_C

<ejecuciones> -> EJECUTAR ID <mas_ejecuciones>

<mas_ejecuciones> -> EJECUTAR ID <mas_ejecuciones> | ε

<exportar> -> EXPORTAR ID <mas_ids>

<mas_ids> -> COMA ID <mas_ids> | ε

Qué método implementa cada no terminal

No terminal	Método	No terminal	Método
<programa>	parsear()	<termino>	termino()
<directivas>	directivas()	<unidad>	unidad()
<agentes>	agentes()	<cargar>	cargar()
<mas_agentes>	masAgentes()	<ejecuciones>	ejecuciones()
<agente>	agente()	<mas_ejecuciones>	masEjecuciones()
<contexto>	contexto()	<exportar>	exportar()
<variables>	variables()	<mas_ids>	masIds()
<comandos>	comandos()	<expresion>	expresion()
<mas_comandos>	masComandos()	<mas_terminos>	masTerminos()
<comando>	comando()	<conectores>	conectores()

Las reglas <mas_...> y <conectores> se implementan como un while en lugar de una llamada recursiva, que es equivalente para una regla recursiva por la derecha.

Hay métodos de apoyo (variable(), ejecucion(), comandoProtegido(), cerrarAgente()) que no son no terminales: agrupan un elemento de una lista o la recuperación de errores.

La gramática es LL(1): en cada decisión basta mirar un token (verifica), y no hay recursividad por la izquierda.

DISEÑO DEL AST (ÁRBOL DE SINTAXIS ABSTRACTA)

El Árbol de Sintaxis Abstracta (AST) es una estructura de datos en forma de árbol que se crea durante el análisis sintáctico para organizar y representar todos los elementos del programa de forma lógica. Su funcionamiento se basa en un nodo principal (Programa) que contiene listas ordenadas con las directivas, los agentes, las ejecuciones y las exportaciones del archivo. A su vez, cada agente agrupa sus propiedades, variables y comandos mediante objetos específicos, lo que permite recorrer y evaluar el código de manera sencilla en las siguientes etapas del compilador.

```java
package com.cunoc.proyecto2_promptzal_analisadorsintactico.ast;
import java.util.List;
/**
 *
 * @author jppax
 */
public class Programa {

    private final List<Directiva> directivas;
    private final List<Agente> agentes;
    private final List<Referencia> ejecuciones;
    private final List<Referencia> exportaciones;

    public Programa(List<Directiva> directivas, List<Agente> agentes, List<Referencia> ejecuciones,
            List<Referencia> exportaciones){
        this.directivas = directivas;
        this.agentes = agentes;
        this.ejecuciones = ejecuciones;
        this.exportaciones = exportaciones;
    }

    public List<Directiva> getDirectivas(){return directivas;}
    public List<Agente> getAgentes(){return agentes;}
    public List<Referencia> getEjecuciones() {return ejecuciones;}
    public List<Referencia> getExportaciones(){return exportaciones;}
}

```

Programa: Es la clase raíz que agrupa todo el archivo, conteniendo listas independientes para las directivas globales, los agentes definidos, las ejecuciones y las exportaciones.

Directiva: Guarda la información básica de las directivas globales del archivo (como el nombre de la directiva, su valor y la fila donde se ubica).

Agente: Representa la estructura de un agente, almacenando su nombre, contexto, la lista de variables declaradas, sus comandos y su posición exacta en el código.

Variable: Modela la declaración de una variable dentro de un agente, guardando su identificador, la expresión asignada y la línea.

Comando: Almacena las órdenes de inteligencia artificial, incluyendo el nombre del comando, su argumento principal, los conectores adicionales, el destino (-> ID) y sus respectivas filas.

Conector: Representa los modificadores o conectores de un comando (como DESDE, EN, SOBRE, COMO), guardando su nombre y la expresión asociada.

Referencia: Se usa para registrar los identificadores sencillos que se emplean en las secciones de ejecución y exportación.

Expresión: Agrupa una lista de términos unidos por el operador de suma (+)

Termino

```java
package com.cunoc.proyecto2_promptzal_analisadorsintactico.ast;

/**
 *
 * @author jppax
 */
public abstract class Termino {

    private final int fila;

    protected Termino(int fila){
        this.fila = fila;
    }
    public int getFila() {
        return fila;
    }
}

```

Termino (Clase abstracta): Sirve como la clase base general para todos los tipos de términos del lenguaje, guardando el número de fila común para el control de posiciones.

TerminoCadena: Representa un valor de texto fijo entre comillas, heredando de Termino y guardando el contenido en su atributo valor.

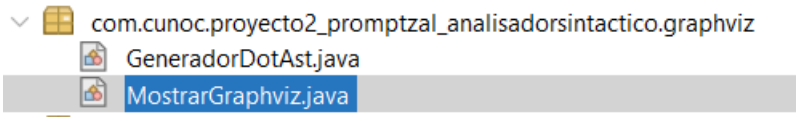
TerminoCargar: Modela la instrucción para cargar archivos externos dentro de las expresiones, almacenando la ruta o nombre del archivo.

TerminoId: Representa el uso de un identificador o variable dentro de una expresión, guardando su nombre y su posición de columna.

TerminoNumero: Modela los valores numéricos enteros o decimales, guardando su valor y opcionalmente su unidad de medida.

GENERACIÓN Y VISUALIZACIÓN GRÁFICA DEL AST

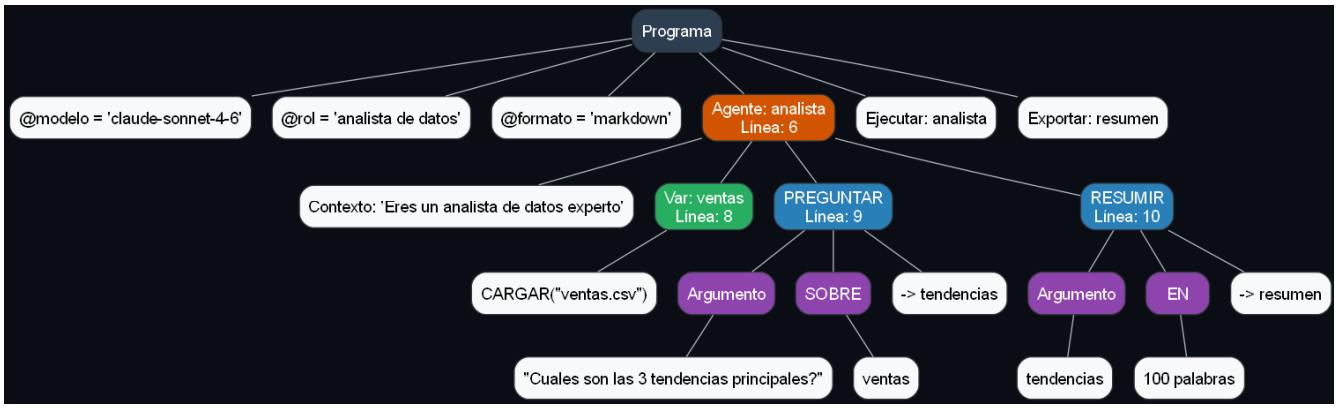
Para visualizar la estructura del árbol de sintaxis abstracta, el compilador recorre los nodos generados desde la raíz (Programa) y construye dinámicamente un archivo de texto con la sintaxis del lenguaje DOT. Este archivo define de forma jerárquica cada nodo, sus propiedades y sus conexiones mediante flechas. Finalmente, el sistema invoca al motor de Graphviz (dot) para compilar dicho archivo de texto y convertirlo automáticamente en una imagen gráfica del árbol.



El método principal generarCodigo(), configura el diseño visual del gráfico (fondo oscuro, orientación de arriba hacia abajo TB, formas redondeadas y colores distintivos) y recorre sucesivamente las listas de directivas, agentes, ejecuciones y exportaciones. A través de llamadas a métodos específicos como agente(), comando(), expresion() y termino(), desglosa recursivamente cada bloque del código fuente hasta llegar a las hojas del árbol, especificando textos, etiquetas y líneas de conexión entre padres e hijos.

```
15      public String generarCodigo(Programa programa) {
16          contador = 0;
17          StringBuilder dot = new StringBuilder();
18
19          dot.append("digraph AST {\n");
20          dot.append("  bgcolor=\"#0a0d14\";\n");
21          dot.append("  rankdir=TB;\n");
22          dot.append("  node [shape=box, style=\"rounded,filled\", fontname=\"Helvetica\", fillcolor=\"#f8f9fa\", co
23          dot.append("  edge [color=\"#adb5bd\", dir=none];\n\n");
24
25          String idRaiz = id();
26          dot.append("  ").append(idRaiz).append(" [label=\"Programa\", fillcolor=\"#2c3e50\", fontcolor=white];\n")
27
28          for (Directiva d : programa.getDirectivas()) {
29              String idDir = id();
30              dot.append("  ").append(idDir).append(" [label=\"").append(d.getNombre()).append(" = '").append(d.getV
31              dot.append("  ").append(idRaiz).append(" -> ").append(idDir).append(";\n");
32          }
33
34          for (Agente a : programa.getAgentes()) {
35              agente(dot, idRaiz, a);
36          }
```

Ejemplo de la generación:



ANALIZADOR SEMÁNTICO

El analizador semántico se encarga de revisar que el código tenga sentido lógico usando una tabla de símbolos, que es básicamente una lista donde registra los agentes y las variables conforme los va encontrando. La lógica es sencilla: recorre todo el programa y guarda cada elemento con su nombre y el lugar al que pertenece (su ámbito) para que las variables de distintos agentes no choquen entre sí. Al mismo tiempo, verifica reglas clave: que no haya agentes repetidos y que no se utilicen variables inventadas que no han sido declaradas previamente. Finalmente, revisa la última parte del archivo para asegurar que la instrucción EJECUTAR llame a agentes que de verdad existen y que EXPORTAR solo pida variables que pertenezcan específicamente a los agentes que acaban de ser ejecutados.

Símbolo

```java
public class Simbolo {

    private final String nombre;
    private final TipoSimbolo tipo;
    private final String ambito;
    private final int fila;

    public Simbolo(String nombre, TipoSimbolo tipo, String ambito, int fila){
        this.nombre = nombre;
        this.tipo = tipo;
        this.ambito = ambito;
        this.fila = fila;
    }

    public String getNombre(){return nombre; }
    public TipoSimbolo getTipo(){return tipo; }
    public String getAmbito(){return ambito;}
    public int getFila(){return fila; }
}
```

La clase Simbolo es simplemente una ficha de registro que guarda los datos básicos de cada elemento del código. Solo almacena cuatro cosas: el nombre, el tipo (si es agente, variable o resultado), el ámbito al que pertenece y la línea donde aparece. Su única función es mantener esta información ordenada para que el analizador pueda consultarla en cualquier momento.

TablaSimbolos

```
5   public class TablaSimbolos {
6       private final List<Simbolo> simbolos = new ArrayList<>();
7
8       public void agregar(Simbolo simbolo) {
9           simbolos.add(simbolo);
10      }
11
12      public boolean existeAgente(String nombre) {
13          for (Simbolo s : simbolos) {
14              if (s.getTipo() == TipoSimbolo.AGENTE && s.getNombre().equals(nombre)) return true;
15          }
16          return false;
17      }
18
19      public boolean existeVariable(String ambito, String nombre) {
20          for (Simbolo s : simbolos) {
21              if (s.getTipo() != TipoSimbolo.AGENTE
22                      && s.getAmbito().equals(ambito) && s.getNombre().equals(nombre)) return true;
23          }
24          return false;
25      }
26
27      public List<Simbolo> getSimbolos() { return simbolos; }
28  }
```

La clase TablaSimbolos funciona como el contenedor de memoria del compilador, utilizando una lista para almacenar todos los objetos Simbolo registrados. Su tarea principal es agregar nuevos elementos y realizar búsquedas dentro de esta lista. Cuenta con métodos específicos para revisar si un agente ya existe buscando por su nombre, y para comprobar si una variable ya está declarada dentro de un ámbito particular, asegurándose de filtrar y buscar solo entre los elementos que no sean de tipo agente.

ATRIBUTO	DESCRIPCIÓN
Nombre	El identificador exacto del agente o la variable.
Tipo	La categoría del elemento. Se clasifica mediante el enumerador TipoSimbolo en tres opciones: AGENTE, VARIABLE_DECLARADA y RESULTADO_COMANDO
Ámbito	El contexto al que pertenece. Los agentes se guardan con el ámbito "global". Las variables usan el nombre de su agente contenedor como ámbito. Esto permite que dos agentes distintos tengan variables con el mismo nombre sin chocar en la memoria.
Fila	El número de línea donde fue declarado en el archivo fuente. No se registra la columna porque el enunciado del proyecto solo pide la línea

Lógica de validación en la memoria

La clase TablaSimbolos cuenta con métodos de búsqueda específicos, como existeAgente y existeVariable. Cuando el analizador semántico revisa una expresión o una instrucción final (como EJECUTAR o EXPORTAR), extrae el nombre del elemento y lo busca en la tabla aplicando el filtro de

su ámbito correspondiente. Si la búsqueda falla porque el elemento no fue registrado, el proceso reporta un error semántico.

TipoSimbolo

```
3    public enum TipoSimbolo {
4        AGENTE("Agente"),
5        VARIABLE_DECLARADA("Variable declarada"),
6        RESULTADO_COMANDO("Resultado de comando");
7
8        private final String etiqueta;
9
10       TipoSimbolo(String etiqueta) {
11           this.etiqueta = etiqueta;
12       }
13
14       public String getEtiqueta() { return etiqueta; }
15   }
```

El enumerador TipoSimbolo funciona como una lista de categorías fijas para clasificar los elementos guardados en la memoria del compilador. Define exactamente tres tipos posibles: agente, variable declarada y resultado de comando.

AnalizadorSemantico

```
    public TablaSimbolos analizar(Programa programa) {
25      for (Agente agente : programa.getAgentes()) {
26          analizarAgente(agente);
27      }
28      validarEjecuciones(programa);
29      validarExportaciones(programa);
30      return tabla;
31  }
```

El método analizar actúa como el director principal del análisis semántico. Su tarea es recibir el programa completo y coordinar el proceso en tres pasos: primero, recorre la lista de todos los agentes para analizar el interior de cada uno; luego, activa la revisión de las ejecuciones; y por último, verifica

las exportaciones. Al finalizar todas estas validaciones, devuelve la tabla de símbolos completa y lista para ser utilizada en la siguiente fase.

Metodos Validacion

```
71      private void validarEjecuciones(Programa programa) {
72          for (Referencia r : programa.getEjecuciones()) {
73              if (!tabla.existeAgente(r.getNombre())) {
74                  errores.add(ErrorCompilacion.semantico(
75                          "EJECUTAR hace referencia a un agente inexistente: '" + r.getNombre() + "'", r.getFila(), r
76              }
77          }
78      }
```

Los métodos de validación comparten una lógica central: todos actúan como filtros de existencia que consultan la tabla de símbolos. Su función principal es extraer el nombre de lo que se intenta utilizar —ya sea una variable dentro de un comando, un agente en la instrucción EJECUTAR, o un dato específico al momento de EXPORTAR— y buscar si ese elemento fue registrado previamente en el contexto adecuado. Si la búsqueda falla y el elemento no existe en la memoria, el método detiene esa acción y genera un error semántico con la fila y columna exactas, asegurando que el programa no procese referencias a cosas que nunca fueron declaradas o que están fuera de su alcance.

GENERACIÓN DEL PROMPTZAL

```
23      public String generar(Programa programa) {
24          StringBuilder sb = new StringBuilder();
25          encabezado(programa, sb);
26          Set<String> escritos = new HashSet<>();
27          for (Referencia ejecucion : programa.getEjecuciones()) {
28              Agente agente = buscarAgente(programa, ejecucion.getNombre());
29              if (agente != null && escritos.add(ejecucion.getNombre())) {
30                  bloqueAgente(agente, sb);
31              }
32          }
33          resultados(programa, sb);
34          return sb.toString();
35      }
```

generar

```
23      public String generar(Programa programa) {
24          StringBuilder sb = new StringBuilder();
25          encabezado(programa, sb);
26          Set<String> escritos = new HashSet<>();
27          for (Referencia ejecucion : programa.getEjecuciones()) {
28              Agente agente = buscarAgente(programa, ejecucion.getNombre());
29              if (agente != null && escritos.add(ejecucion.getNombre())) {
30                  bloqueAgente(agente, sb);
31              }
32          }
33          resultados(programa, sb);
34          return sb.toString();
35      }
```

Es el encargado de armar el documento final uniendo el encabezado, los agentes y los resultados. Utiliza una lista especial (HashSet) para recordar qué agentes ya fueron escritos, evitando que se impriman repetidos si el usuario los ejecuta varias veces en el código.

Instrucción

```
    private String instruccion(Comando c) {
86      String x = texto(c.getArgumento());
87      Expresion sobre = conector(c, "SOBRE");
88      Expresion desde = conector(c, "DESDE");
89      Expresion en = conector(c, "EN");
90      Expresion como = conector(c, "COMO");
91      String guarda = "el resultado";
92      String base;
        switch (c.getNombre()) {
```

Es el núcleo de la traducción. Usa una estructura `switch` para identificar el tipo de comando (como PREGUNTAR o GENERAR) y armar una frase en lenguaje natural. Si detecta un comando que no está programado, crea una frase genérica para que el programa no falle.

Métodos texto y origen: Se encargan de extraer y formatear los valores de las variables. Su función más importante es que, al detectar que un valor viene de un archivo, le agregan automáticamente el texto "(adjuntar el archivo)" para que la instrucción final tenga sentido.

```
159     private String texto(Termino termino) {
160         if (termino instanceof TerminoCadena cadena) return cadena.getValor();
161         if (termino instanceof TerminoId id) return id.getNombre();
162         if (termino instanceof TerminoNumero numero) {
163             return numero.getUnidad() == null ? numero.getValor() : numero.getValor() + " " + numero.getUnidad();
164         }
165         if (termino instanceof TerminoCargar carga) return "el contenido del archivo \"" + carga.getArchivo() + "\"
166         return "";
167     }
168
169     private String origen(Expresion expresion) {
170         List<String> partes = new ArrayList<>();
171         for (Termino t : expresion.getTerminos()) {
172             partes.add(origen(t));
173         }
174         return String.join(" + ", partes);
175     }
176
```

Ejemplo

A continuación se muestra como el analizador toma un ejemplo del lenguaje y el prompt que genera .

```
@modelo "claude-sonnet-4-6"
@rol "analista de datos"
@formato "markdown"
// Agente que prepara el analisis de un archivo de ventas
AGENTE analista {
contexto = "Eres un analista de datos experto"
variable ventas = CARGAR ("ventas.csv")
PREGUNTAR "Cuales son las 3 tendencias principales?" SOBRE ventas -> tendencias
RESUMIR tendencias EN 100 palabras -> resumen
}
EJECUTAR analista
EXPORTAR resumen
```

Prompt que el programa genera:

```
# Prompt generado con PromptZal
Modelo destino: claude-sonnet-4-6
Rol general: analista de datos
Formato de respuesta: markdown
## Agente: analista
Contexto: Eres un analista de datos experto
Variables disponibles:
- ventas: contenido del archivo "ventas.csv" (adjuntar el archivo)
Instrucciones:
1. Basandote en ventas, responde: Cuales son las 3 tendencias principales? Guarda la respuesta como tendencias.
2. Resume tendencias en un maximo de 100 palabras. Guarda el resultado como resumen.
## Resultado solicitado
- resumen
```

COMPILADOR

La clase Compilador funciona como el director general del programa. Su método compilar recibe el texto del usuario y ejecuta las cuatro fases en estricto orden: primero llama al analizador léxico para obtener los tokens, luego pasa esos tokens al parser para construir el árbol sintáctico (AST), después envía el árbol al analizador semántico para crear la tabla de símbolos y realizar validaciones, y finalmente utiliza el generador para crear el texto final. Todo este proceso comparte una misma lista de errores que recolecta los fallos de las tres primeras fases; al terminar, la clase ordena estos errores por fila y columna, y empaqueta todos los resultados en un objeto ResultadoCompilacion para enviarlos a la interfaz gráfica.

```
15  public class Compilador {
16
17      public ResultadoCompilacion compilar(String texto) {
18          AnalizadorLexico lexico = new AnalizadorLexico();
19          lexico.analizar(texto);
20
21          List<ErrorCompilacion> errores = new ArrayList<>(lexico.getErrores());
22          Programa programa = new Parser(lexico.getTokens(), errores).parsear();
23          TablaSimbolos tabla = new AnalizadorSemantico(errores).analizar(programa);
24          String promptzal = new GeneradorPromptzal().generar(programa);
25
26          errores.sort(Comparator.comparingInt(ErrorCompilacion::getFila)
27          .thenComparingInt(ErrorCompilacion::getColumna));
28
29          return new ResultadoCompilacion(lexico.getTokens(), errores, programa, tabla, promptzal);
30      }
31
32          public List<Token> tokenizar(String texto) {
33          AnalizadorLexico lexico = new AnalizadorLexico();
34          lexico.analizar(texto);
35          return lexico.getTokens();
36      }
37  }
```

```java
public class ResultadoCompilacion {

    private final List<Token> tokens;
    private final List<ErrorCompilacion> errores;
    private final Programa programa;
    private final TablaSimbolos tabla;
    private final String promptzal;

    public ResultadoCompilacion(List<Token> tokens, List<ErrorCompilacion> errores,
                                Programa programa, TablaSimbolos tabla, String promptzal) {
        this.tokens = tokens;
        this.errores = errores;
        this.programa = programa;
        this.tabla = tabla;
        this.promptzal = promptzal;
    }

    public List<Token> getTokens() { return tokens; }
    public List<ErrorCompilacion> getErrores() { return errores; }
    public Programa getPrograma() { return programa; }
    public TablaSimbolos getTabla() { return tabla; }
    public String getPromptzal() { return promptzal; }
}
```

REPORTES HTML

El compilador no solo produce el archivo final, sino que también genera reportes detallados en formato HTML para facilitar la depuración y el análisis del código fuente. Estos reportes incluyen la lista de errores encontrados, la tabla de símbolos registrada y la vista general de los tokens.

Para la generación de los reportes, se diseño la clase Html, la cual actúa como una utilidad centralizada para construir la estructura y el estilo visual de las páginas.

```
    private Html() {
    }

    static String escapar(String texto) {
        if (texto == null) return "";
        return texto.replace("&", "&amp;")
                    .replace("<", "&lt;")
                    .replace(">", "&gt;")
                    .replace("\"", "&quot;");
    }

    static String celda(String texto) {
        return "<td>" + escapar(texto) + "</td>";
    }

    static String celdaPos(String texto) {
        return "<td class=\"pos\">" + escapar(texto) + "</td>";
    }

    static String celdaCodigo(String texto) {
        return "<td class=\"codigo\">" + (texto == null ? "-" : escapar(texto)) + "</td>";
    }

    static String celdaToken(String texto) {
        return "<td class=\"codigo token\">" + (texto == null ? "-" : escapar(texto)) + "</td
    }
```

CONCLUSIÓN

El compilador de PromptZal unifica todas las fases de análisis para transformar un archivo de entrada en un prompt funcional. El flujo avanza paso a paso: primero extrae los tokens, luego organiza la estructura con un árbol sintáctico a mano y verifica la validez de las variables y agentes mediante una tabla de símbolos. Finalmente, traduce las instrucciones a lenguaje natural y documenta todo el proceso generando reportes en HTML y diagramas visuales.